MECHATRONICS AND IOT

ASSIGNMENT - 11

GPS-Based Vehicle Tracking and Fleet Management System

TEAM MEMBERS:

JEEVA T	**24MER020**
NITHEESH M	**24MER036**
SANJAY S	**24MER054**
SIVA BALAJI S	**24MER058**
SIVA S	**24MER059**

1. Project Overview

The GPS-Based Vehicle Tracking and Fleet Management System is an IoT-based system designed to monitor and track vehicles in real time. The system uses a **GPS module** to obtain the vehicle's geographical location, while a **microcontroller** processes the GPS data and sends it to a remote monitoring platform through a wireless communication module.

The system helps fleet owners and transportation managers monitor vehicle location, movement, and operational status remotely. The current location of the vehicle can be viewed on a mobile phone or computer using a web-based or IoT monitoring platform.

The proposed system improves fleet management by reducing manual monitoring, improving vehicle security, optimizing transportation operations, and providing useful location information to the fleet operator.

Main components:

- ESP32 Development Board
- NEO-6M GPS Module
- Li-ion Battery
- LED-based status indication.
- Jumper Wires
- Breadboard.
- LED + 220 Ω Resistor.
- Buzzer.

2. Project Statement

In conventional transportation systems, vehicle monitoring is often performed manually through telephone communication or periodic reporting from drivers. This method does not provide continuous information about the exact location of vehicles. It can also make it difficult for fleet managers to identify delays, unauthorized vehicle movement, or inefficient routes.

Therefore, there is a need for a low-cost and reliable system that can continuously track vehicles and provide their real-time location to the fleet operator.

The proposed GPS-Based Vehicle Tracking and Fleet Management System solves this problem by integrating GPS technology, a microcontroller, and wireless communication. The system collects the vehicle's latitude and longitude coordinates and transmits them to a remote monitoring platform

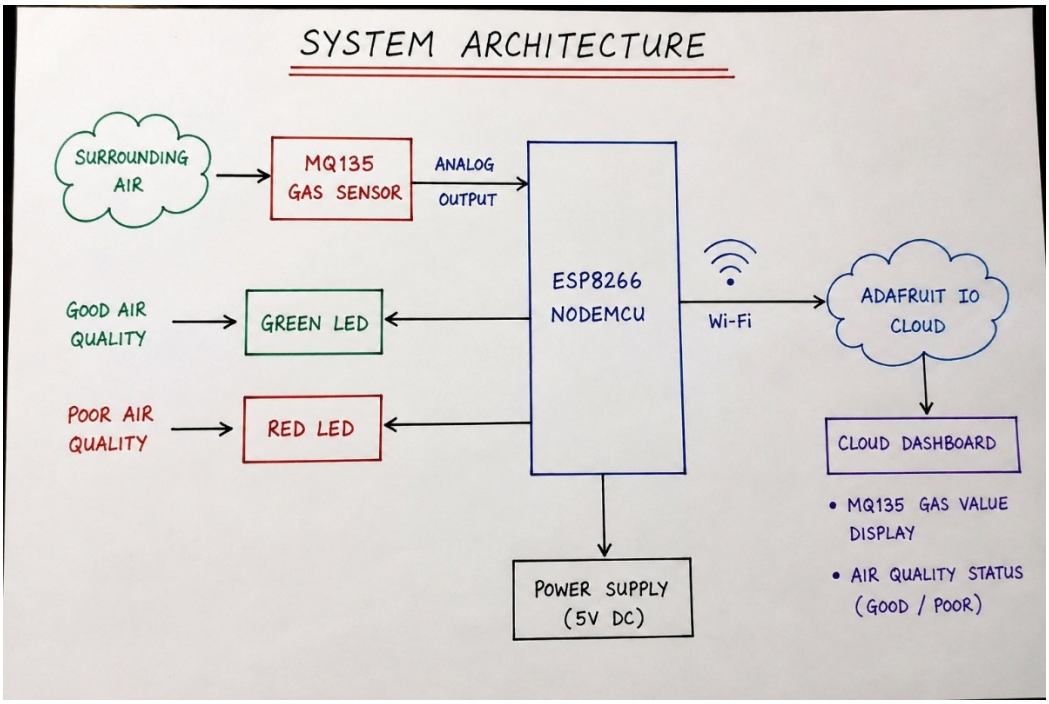
3. Proposed Solution

The proposed system consists of a GPS receiver, ESP32 microcontroller, and wireless communication facility. The GPS module receives signals from satellites and calculates the current latitude, longitude, speed, and time information.

The system operates as follows:

- To track vehicle location in real time.
- To obtain accurate latitude and longitude coordinates.
- To transmit vehicle information wirelessly.
- To monitor vehicle movement remotely.
- To improve fleet management efficiency.
- To provide a low-cost vehicle tracking solution.
- To improve vehicle security and monitoring.

4. System Architecture:

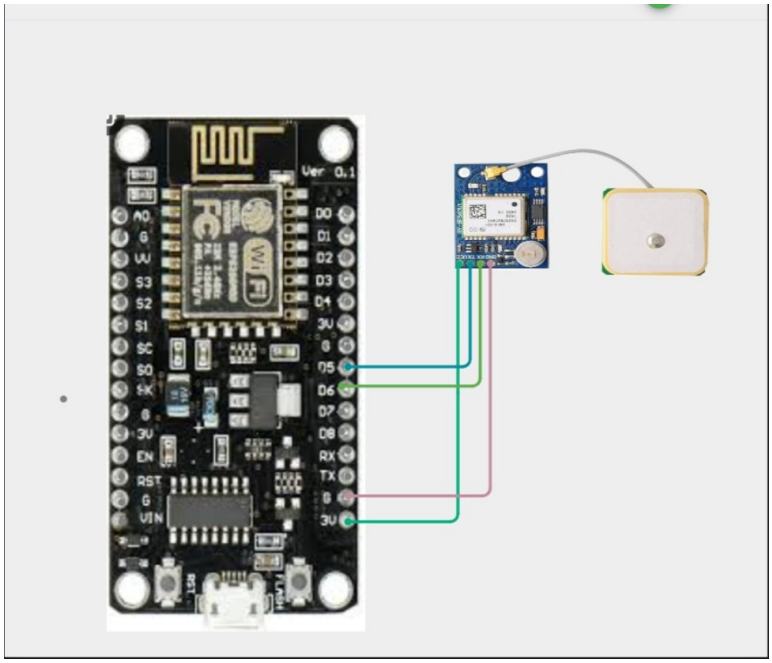


5. Circuit Table:

CIRCUIT TABLE

S. No.	Component	Pin	ESP32 Pin	Description
1.	NEO-6M GPS Module	VCC	5V / VIN	Power Supply (5V)
		GND	GND	Ground
		TX	GPIO16 (RX2)	GPS TX to ESP32 RX
		RX	GPIO17 (TX2)	GPS RX to ESP32 TX
2.	ESP32 NodeMCU	VIN / 5V	–	Power Input (5V)
		GND	–	Ground
		EN / RST	–	Enable / Reset (Optional)
3.	Buzzer	+ (Positive)	GPIO26	Buzzer Control
		– (Negative)	GND	Ground
4.	Green LED	Anode (+)	GPIO27 through 220Ω	Green LED (Power / Safe)
		Cathode (–)	GND	Ground
5.	Red LED	Anode (+)	GPIO14 through 220Ω	Red LED (Alert / Unsafe)
		Cathode (–)	GND	Ground
6.	Power Supply	+5V	5V / VIN	5V DC Supply
		GND	GND	Ground
7.	Wi-Fi	Built-in	ESP32 Wi-Fi Module	Internet Connection
8.	IoT / Cloud Platform	–	Internet	Cloud Dashboard / Monitoring

6. Circuit Design:



7. Algorithm:

Step-by-step algorithm:

Step 1: Start the system.

Step 2: Initialize the ESP32.

Step 3: Initialize serial communication.

Step 4: Initialize the GPS module.

Step 5: Connect ESP32 to the Wi-Fi network.

Step 6:Wait for GPS satellite signals.

Step 7: Read GPS data.

Step 8: Check whether valid GPS coordinates are available.

Step 9: Extract latitude and longitude.

Step 10: Extract additional information such as speed and time if available.

Step 11: Send the location data to the IoT/cloud platform.

Step 12: Display/update the vehicle location.

Step 13: Repeat the process continuously.

Step 14: If GPS data is unavailable, wait and attempt to acquire the signal again.

Step 15: Stop the system.

8. Coding:

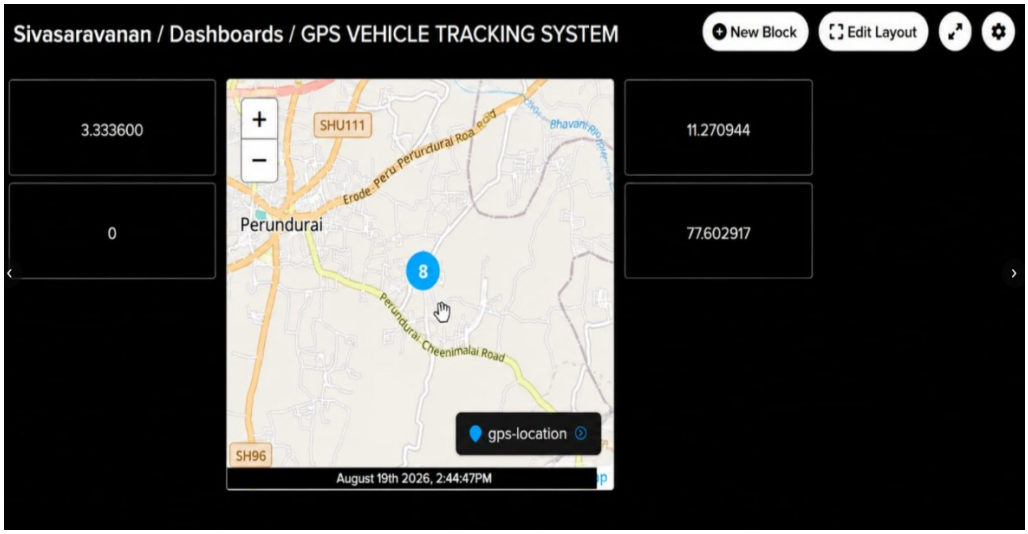
```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <TinyGPSPlus.h>
#include <SoftwareSerial.h
// ---------------- WiFi ----------------
#define WIFI_SSID "OnePlus Nord CE5 7C5C"
#define WIFI_PASS "qkts5737
// ------------ Adafruit IO -------------
#define IO_USERNAME "Sivasaravanan"
#define IO_KEY "aio_ADVS50c7oG5Sx1R3FskFAq0fjUrB"
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS
// -------------- Feeds -----------------
AdafruitIO_Feed *latitudeFeed  = io.feed("latitude");
AdafruitIO_Feed *longitudeFeed = io.feed("longitude");
AdafruitIO_Feed *speedFeed     = io.feed("speed");
AdafruitIO_Feed *satelliteFeed = io.feed("satellites");
AdafruitIO_Feed *locationFeed  = io.feed("gps-location")
// ------------ GPS Pins ----------------
// GPS TX -> D5 (GPIO14)
// GPS RX -> D6 (GPIO12
```

```
SoftwareSerial gpsSerial(D5, D6);
TinyGPSPlus gps;
// Upload every 20 seconds
const unsigned long uploadInterval = 20000;
unsigned long lastUpload = 0;
void setup()
  Serial.begin(115200);
  gpsSerial.begin(9600);
  Serial.println();
  Serial.println("====================================");
  Serial.println(" GPS VEHICLE TRACKING SYSTEM");
  Serial.println("====================================");
  Serial.print("Connecting to Adafruit IO")
  io.connect();
  while (io.status() < AIO_CONNECTED) {
    Serial.print(".");
    delay(500);
  }
  Serial.println("\nConnected to Adafruit IO.");
  Serial.println("Waiting for GPS Fix...")
void loop() {
  io.run()
  while (gpsSerial.available()) {
    gps.encode(gpsSerial.read());
  if (!gps.location.isValid())
    static unsigned long lastMessage = 0;
    if (millis() - lastMessage > 5000) {
      lastMessage = millis();
      Serial.println("Waiting for GPS satellites...");
```

```
  }
  return;
}
if (millis() - lastUpload >= uploadInterval)
  lastUpload = millis()
  double latitude   = gps.location.lat();
  double longitude  = gps.location.lng();
  double speed      = gps.speed.kmph();
  int satellites    = gps.satellites.value();
  double altitude   = gps.altitude.isValid() ? gps.altitude.meters() : 0;
  Serial.println("---------------------------------");
  Serial.print("Latitude   : ");
  Serial.println(latitude, 6);
  Serial.print("Longitude  : ");
  Serial.println(longitude, 6);
  Serial.print("Speed      : ");
  Serial.print(speed, 2);
  Serial.println(" km/h");
  Serial.print("Satellites : ");
  Serial.println(satellites);
  Serial.print("Altitude   : ");
  Serial.print(altitude);
  Serial.println(" m");
  Serial.println("Uploading to Adafruit IO...");
  latitudeFeed->save(latitude);
  delay(200);
  longitudeFeed->save(longitude);
  delay(200);
  speedFeed->save(speed);
```

```
    delay(200);
    satelliteFeed->save(satellites);
    delay(200);
    locationFeed->save(speed, latitude, longitude, altitude);
    Serial.println("Upload Successful.");
    Serial.println("--------------------------------");
  }
}
```

9. System Working :



When the system is powered ON, the ESP32 initializes the GPS module and establishes a wireless Internet connection. The GPS receiver communicates with satellites and receives positioning signals.

The ESP32 processes the received GPS information and sends the location data through Wi-Fi to the selected IoT/cloud platform. The fleet operator can then view the vehicle's current location remotely.

The system continuously repeats this process at regular intervals. Therefore, the fleet manager can monitor vehicle movement without requiring direct communication with the driver.

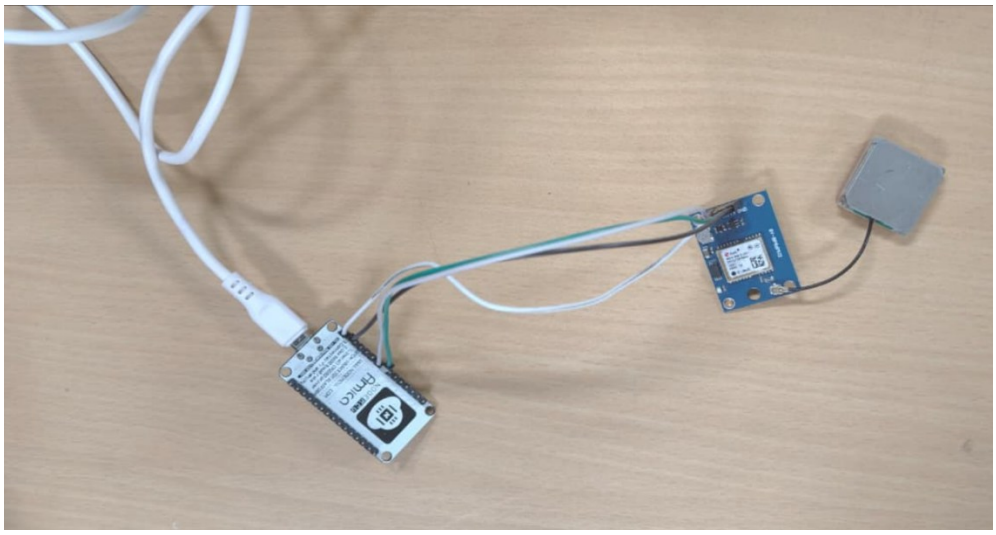
10. BILL OF MATERIAL:

BILL OF MATERIAL

S.No.	Component		Specification / Description	Quantity
1.		**ESP32 NodeMCU**	ESP32 Wi-Fi Development Board	1
2.		**NEO-6M GPS Module**	GPS Module with Antenna (3.3V – 5V)	1
3.		**MQ135 Gas Sensor**	Gas Sensing Module (Analog Output)	1
4.		**Green LED**	5mm LED (Green)	1
5.		**Red LED**	5mm LED (Red)	1
6.		**Resistor**	220 Ω Resistor (for LEDs)	2
7.		**Breadboard**	Solderless Breadboard	1
8.		**Jumper Wires**	Male to Male Jumper Wires	As required
9.		**USB Cable**	Micro USB Cable for Power & Programming	1
10.		**Wi-Fi & Adafruit IO Cloud**	Internet Connection & Cloud Platform for Data Monitoring	1

Reference: *Based on project – GPS-Based Vehicle Tracking and Fleet Management System*

11. Prototype Implementation:



The prototype of the GPS-Based Vehicle Tracking and Fleet Management System was developed by integrating an ESP32 NodeMCU, NEO-6M GPS module, OLED display, power supply, and jumper wires. The prototype was designed to demonstrate real-time vehicle location tracking in a simple and low-cost manner.

First, the ESP32 NodeMCU was selected as the main controller because it provides sufficient processing capability and built-in Wi-Fi connectivity. The NEO-6M GPS module was connected to the ESP32 through serial communication. The GPS module receives signals from satellites and provides the vehicle's latitude, longitude, speed, date, and time information.

During prototype testing, the GPS module was placed in an open area to obtain better satellite reception. Initially, the GPS module may require some time to obtain its first satellite fix. Once the GPS signal becomes valid, the latitude and longitude values are continuously updated.

12. Results and Performance Analysis:

The developed GPS-Based Vehicle Tracking and Fleet Management System was successfully implemented and tested using an ESP32 controller and NEO-6M GPS module. During testing, the GPS module was able to receive satellite signals and provide the geographical coordinates of the vehicle. The

obtained latitude and longitude values were processed by the ESP32 and displayed through the monitoring interface.

The system demonstrated the ability to continuously update the vehicle's location at regular intervals. When the vehicle was moved from one location to another, the GPS coordinates changed accordingly, allowing the movement of the vehicle to be monitored remotely. This confirms that the proposed system can be effectively used for basic real-time vehicle tracking applications.

The wireless communication capability of the ESP32 enabled the collected GPS information to be transmitted to an IoT/cloud platform. This reduced the need for manual vehicle status reporting and allowed the fleet operator to monitor the vehicle remotely. The system also showed that GPS-based tracking can provide useful information for fleet management, route monitoring, and vehicle security.

Performance Analysis:

The prototype performed successfully in the following areas

The major performance parameters observed during testing include GPS location accuracy, response time, wireless connectivity, data update rate, and system reliability. GPS accuracy depends on satellite visibility, environmental conditions, and the location of the vehicle. In open outdoor areas, the GPS module generally provides better positioning performance, while buildings, trees, and other obstacles may increase the time required to obtain a GPS fix.

The system showed satisfactory performance for prototype-level vehicle tracking. The ESP32 successfully received GPS data and processed the coordinates without significant delay. Wireless data transmission was also reliable when a stable Wi-Fi connection was available. However, the system's Internet-based monitoring range depends on the availability of Wi-Fi or another communication network.

The prototype can be further improved by adding GSM/4G communication, allowing the vehicle to transmit location information even when Wi-Fi is unavailable. Additional features such as geofencing, route history, overspeed alerts, fuel monitoring, driver identification, and emergency alerts can also be integrated.

The project successfully demonstrates that an ESP32 and GPS-based IoT system can provide an economical solution for vehicle tracking and basic fleet management. The prototype provides real-time location information, remote monitoring capability, and a foundation for developing a more advanced commercial fleet management system.